Compiladores e Intérpretes

Analizador Sintáctico – Reglas de Sintaxis e Intuiciones

Universidad Nacional del Sur
Departamento de Ciencias e Ingeniería de la Computación
2026

Contexto

- Para entender y controlar la **estructura** de un programa fuente hay que analizar si sigue las reglas de **sintaxis** del lenguaje
- Estas reglas están expresadas en términos de **tokens**, mientras que el fuente es una cadena de caracteres
- El **Analizador Léxico** es el encargado armar los **tokens**



Contexto

- La tarea del **Analizador Sintáctico** será controlar que la **cadena de tokens** resultante del Léxico sea **válida** según la **sintaxis** del lenguaje

- ¿Qué necesitamos para realizar esta tarea?
  - La **especificación** las reglas de sintaxis
  - Un método para **identificar** las cadenas de tokens válidas

Reglas de Sintaxis

Reglas de Sintaxis

- Para **especificar la sintaxis** se utilizan **lenguajes formales**
- ¿Qué lenguaje formal usamos?
  - Por **eficiencia y simplicidad** tratamos de usar **el más "chico" posible**
- ¿Por qué no los lenguajes regulares?
  - **No pueden expresar anidamiento**
    - Por ejemplo, controlar ( ) balanceados: () (()) ((())), etc.
- Usamos los **Lenguajes Libres de Contexto**

Un autómata finito no puede recordar la cantidad de veces que pasó por un estado

Los **lenguajes libres de contexto** son generados por las **gramáticas libres de contexto**

Reglas de Sintaxis

- **Formalmente** una Gramática Libre de Contexto es (T,NT,I,P) donde:
  - Un conjunto de terminales T
  - Un conjunto de no terminales NT
  - Un símbolo de NT inicial I
  - Un conjunto de producciones P de la forma:

$$A \rightarrow b_1 b_2 \ldots b_n$$

Donde los b_i son terminales de T, no terminales de NT o vacío ε



Reglas de Sintaxis

- **Formalmente** una Gramática Libre de Contexto es (T,NT,I,P) donde:
  - Un conjunto de terminales T
  - Un conjunto de no terminales NT
  - Un símbolo de NT inicial I
  - Un conjunto de producciones P de la forma:

$$A \rightarrow b_1 b_2 \ldots b_n$$

Donde los b_i son terminales de T, no terminales de NT o vacío ε

Los no terminales los notamos con mayúsculas y los terminales con símbolos y minúsculas

Producciones Abreviadas:

$A \rightarrow \beta_1 \ldots \beta_n \mid \gamma_1 \ldots \gamma_n$

Es equivalente a

$A \rightarrow \beta_1 \ldots \beta_n$

$A \rightarrow \gamma_1 \ldots \gamma_n$



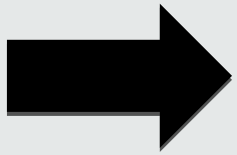
Reglas de Sintaxis

- Cadena derivable: partiendo del símbolo inicial I aplicamos producciones y llegamos a la cadena

Una **cadena derivable** en una gramática G pertenece al **lenguaje** de G.

Aplicar producción: reemplazar NT por la parte derecha de una producción que lo tenga como lado izquierdo.

Expresiones Básicas
E → E + E
E → (E)
E → id
E → num



((id+num)+id)



Reglas de Sintaxis

- **Forma sentencial:** es una cadena de símbolos (T o NT) de la gramática, si es un paso intermedio de una derivación valida

Por ejemplo: ((id+E)+E) para la gramática anterior



Reglas de Sintaxis

- **Forma sentencial:** es una cadena de símbolos (T o NT) de la gramática, si es un paso intermedio de una derivación valida

Por ejemplo: ((id+E)+E) para la gramática anterior

- **Derivación a Izquierda:** expandimos NT de más a la izquierda

$$E \Rightarrow (E) \Rightarrow (E+E) \Rightarrow ((E)+E) \Rightarrow ((E+E)+E) \Rightarrow ((id+E)+E) \Rightarrow ((id+num)+E) \Rightarrow ((id+num)+id)$$



Reglas de Sintaxis

- **Forma sentencial:** es una cadena de símbolos (T o NT) de la gramática, si es un paso intermedio de una derivación valida

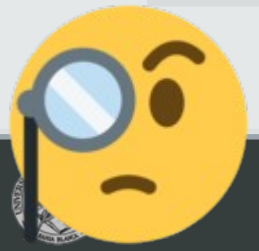
Por ejemplo: ((id+E)+E) para la gramática anterior

- **Derivación a Izquierda:** expandimos NT de más a la izquierda

$$E \Rightarrow (E) \Rightarrow (E+E) \Rightarrow ((E)+E) \Rightarrow ((E+E)+E) \Rightarrow ((id+E)+E) \Rightarrow ((id+num)+E) \Rightarrow ((id+num)+id)$$

- **Derivación a Derecha:** expandimos NT de más a la derecha

$$E \Rightarrow (E) \Rightarrow (E+E) \Rightarrow (E+id) \Rightarrow ((E)+id) \Rightarrow ((E+E)+id) \Rightarrow ((E+num)+id) \Rightarrow ((id+num)+id)$$



Construyendo Analizadores Sintácticos

- Nosotros ya tenemos la cadena, es decir el programa fuente…

¿Cómo seguimos?

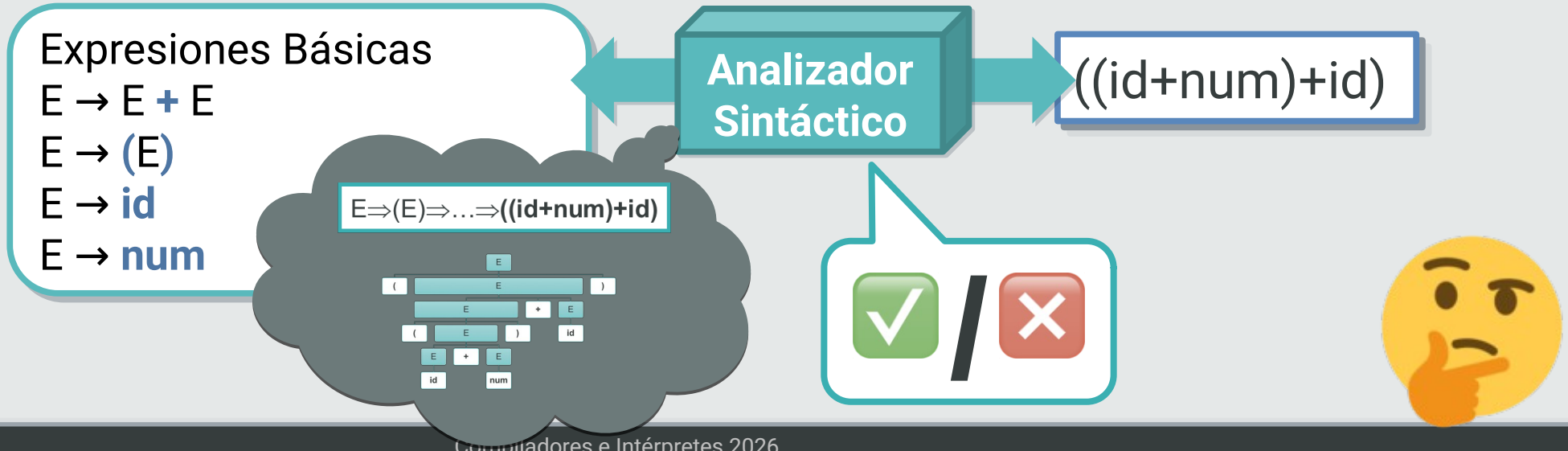


Construyendo Analizadores Sintácticos

- Nosotros ya tenemos la cadena, es decir el programa fuente…

¿Cómo seguimos?

- A partir de la **cadena** reconstruir la **derivación** (implícitamente) analizando qué **producciones** se hubiesen **aplicado** para llegar a ella.



Construyendo Analizadores Sintácticos

- **Estrategia exhaustiva:** pruebo todas las combinaciones de producciones que me lleven a la cadena.

Muy costoso! Es del orden de la cantidad de producciones elevado a la cantidad de tokens de la cadena!



Construyendo Analizadores Sintácticos

- **Estrategia exhaustiva:** pruebo todas las combinaciones de producciones que me lleven a la cadena.

Muy costoso! Es del orden de la cantidad de producciones elevado a la cantidad de tokens de la cadena!



- Utilizar una estrategia más sofisticada
- Elegir producciones ni bien sea conveniente a medida que solicitamos tokens

Construyendo Analizadores Sintácticos

En los subtemas que siguen vamos a ver las dos principales estrategias:

- **Descendente (Top-Down)**: simula el proceso de derivación partiendo desde el primer no terminal y trata de llegar a la cadena.
- **Ascendente (Bottom-Up):** simula el proceso de derivación de manera inversa y reduce la cadena hasta llegar al NT inicial

Compiladores e Intérpretes
Analizador Sintáctico – Descendente – Intuiciones

Universidad Nacional del Sur
Departamento de Ciencias e Ingeniería de la Computación
2026

Idea General

- Considerando el **programa fuente** (cadena) reconstruir la **derivación** (implícitamente) analizando qué **producciones** se hubiesen **aplicado** para llegar a ella.

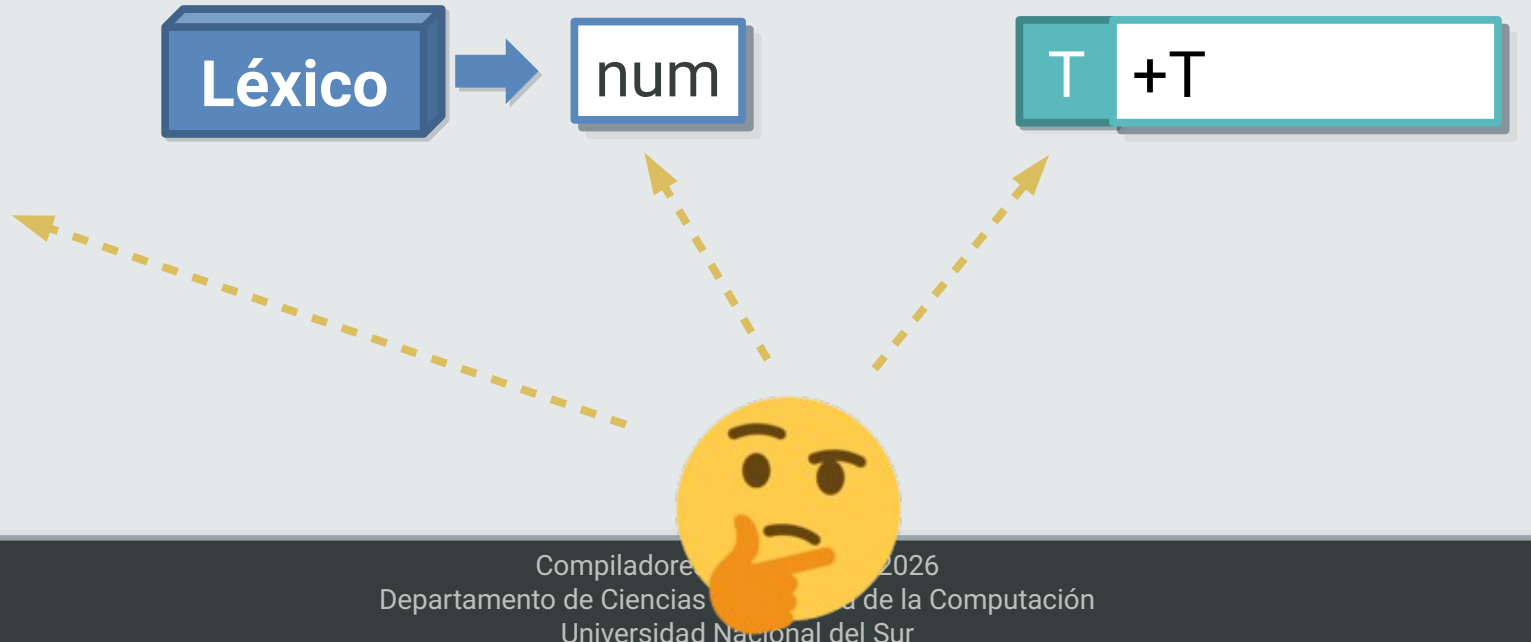
En un analizador descendente:

- Se simula el proceso de derivación partiendo desde el primer no terminal y trata de llegar a la cadena.

Sintáctico Descendente - Intuición

- Intuición: **Simular** la derivación a izquierda
  - Emular la forma sentencial de cada paso de la derivación, analizando el **símbolo** (T o NT) **de más a la izquierda** y el **token actual** enviado por el Léxico

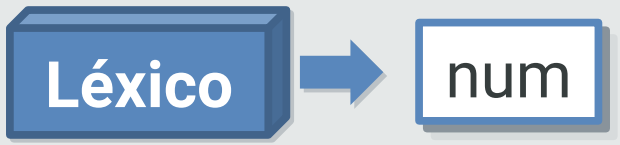
E → E + T
E → T
T → (E)
T → id
T → num

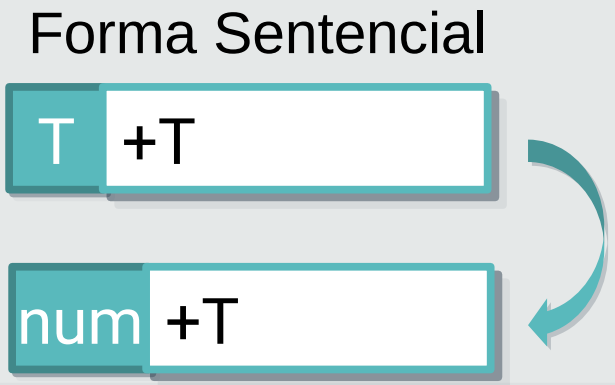


Sintáctico Descendente - Intuición

- Intuición: **Simular** la derivación a izquierda
  - Emular la forma sentencial de cada paso de la derivación, analizando el **símbolo** (T o NT) **de más a la izquierda** y el **token actual** enviado por el Léxico
  - Si el símbolo de más a la izquierda de la forma es:
    - **NT**: **elegir** una **producción** para ese no terminal y **reemplazarlo por la parte derecha**

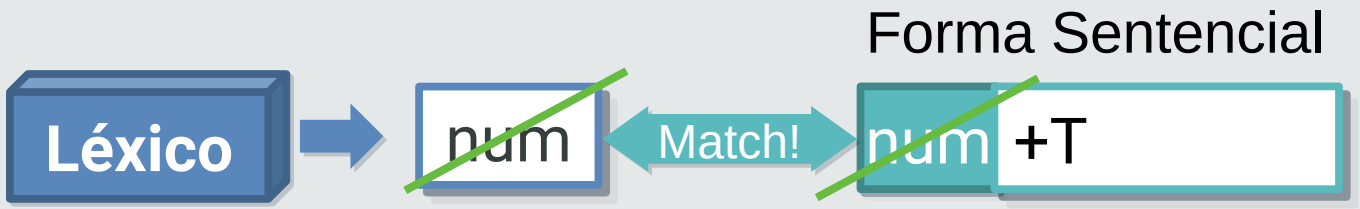
E → E + T
E → T
T → (E)
T → id
T → num





Sintáctico Descendente - Intuición

- Intuición: **Simular** la derivación a izquierda
  - Emular la forma sentencial de cada paso de la derivación, analizando el **símbolo** (T o NT) **de más a la izquierda** y el **token actual** enviado por el Léxico
  - Si el símbolo de más a la izquierda de la forma es:
    - **NT**: **elegir** una **producción** para ese no terminal y **reemplazarlo por la parte derecha**
    - **T**: ver que "**matchee**" con el token actual, en caso de hacerlo **remover ese T** y solicitar un **nuevo token**



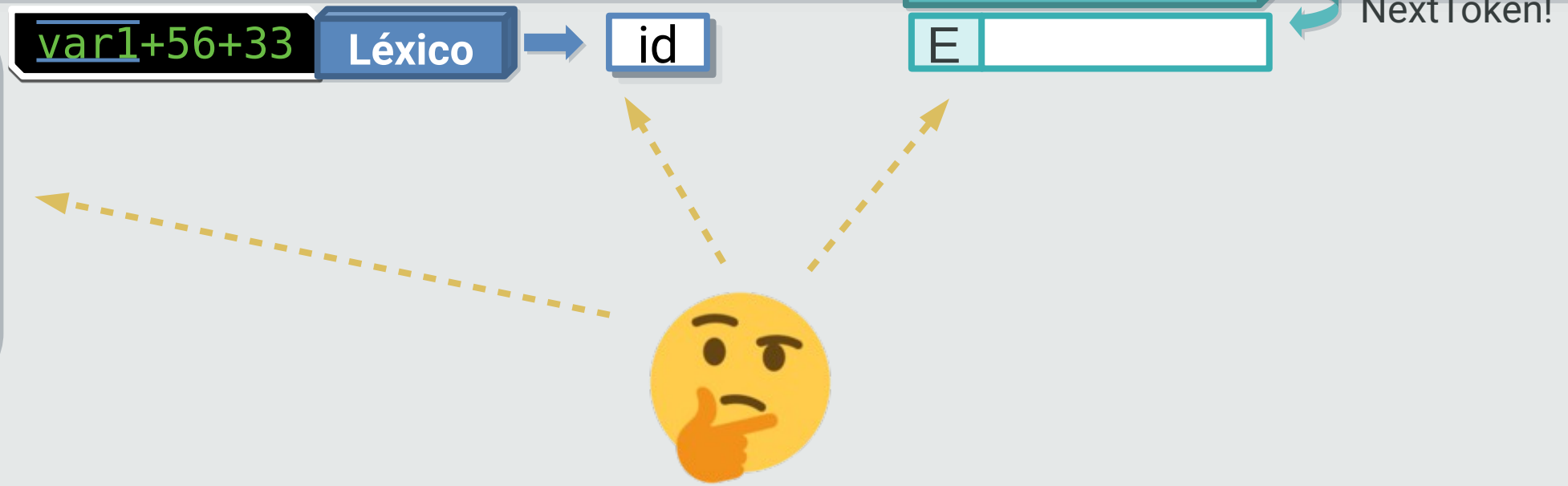
Sintáctico Descendente - Intuición

- Intuición: **Simular** la derivación a izquierda
  - Emular la forma sentencial de cada paso de la derivación, analizando el **símbolo** (T o NT) **de más a la izquierda** y el **token actual** enviado por el Léxico
  - Si el símbolo de más a la izquierda de la forma es:
    - **NT**: **elegir** una **producción** para ese no terminal y **reemplazarlo por la parte derecha**
    - **T**: ver que "**matchee**" con el token actual, en caso de hacerlo **remover ese T** y solicitar un **nuevo token**

En vez de llegar a la cadena, **arrancamos del NT inicial** y vamos a ir **consumiendo** los **tokens actuales** del Léxico cuando corresponda y por lo tanto **reconoceremos** la cadena cuando lleguemos a la **forma vacía**

Sintáctico Descendente – Intuición – Ejemplo

E → E + T
E → T
T → (E)
T → id
T → num



Sintáctico Descendente – Intuición – Ejemplo

Próxima!

E → E + T
E → T
T → (E)
T → id
T → num





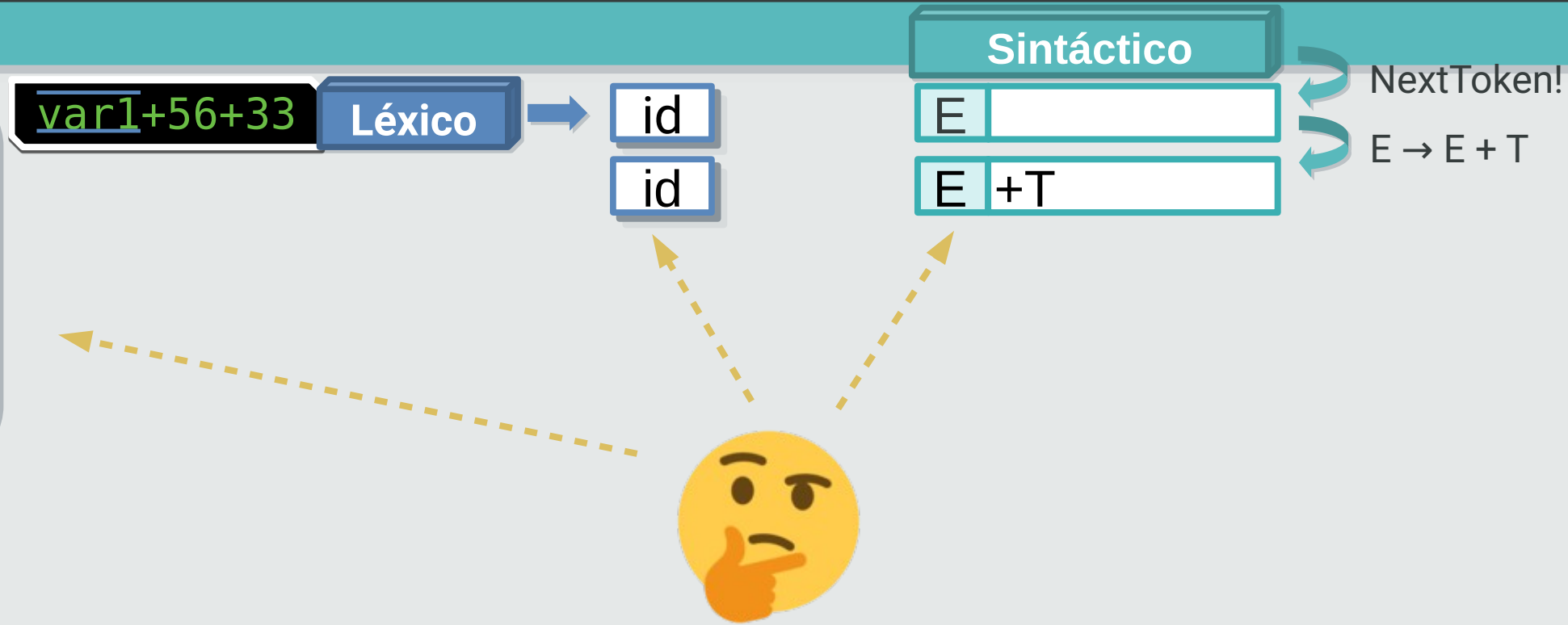


Supongamos que somos nosotros los que tenemos que elegir producción aplicar cuando hay un NT en la izquierda de la forma sentencial

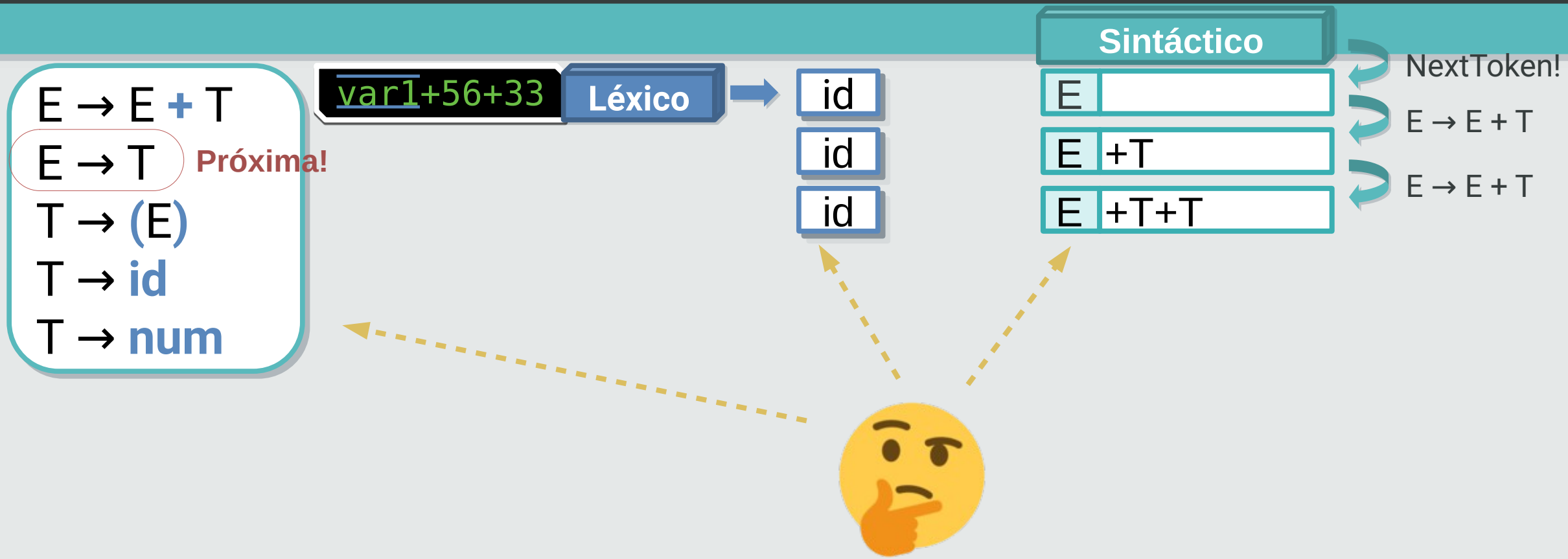
Sintáctico Descendente – Intuición – Ejemplo

Próxima!

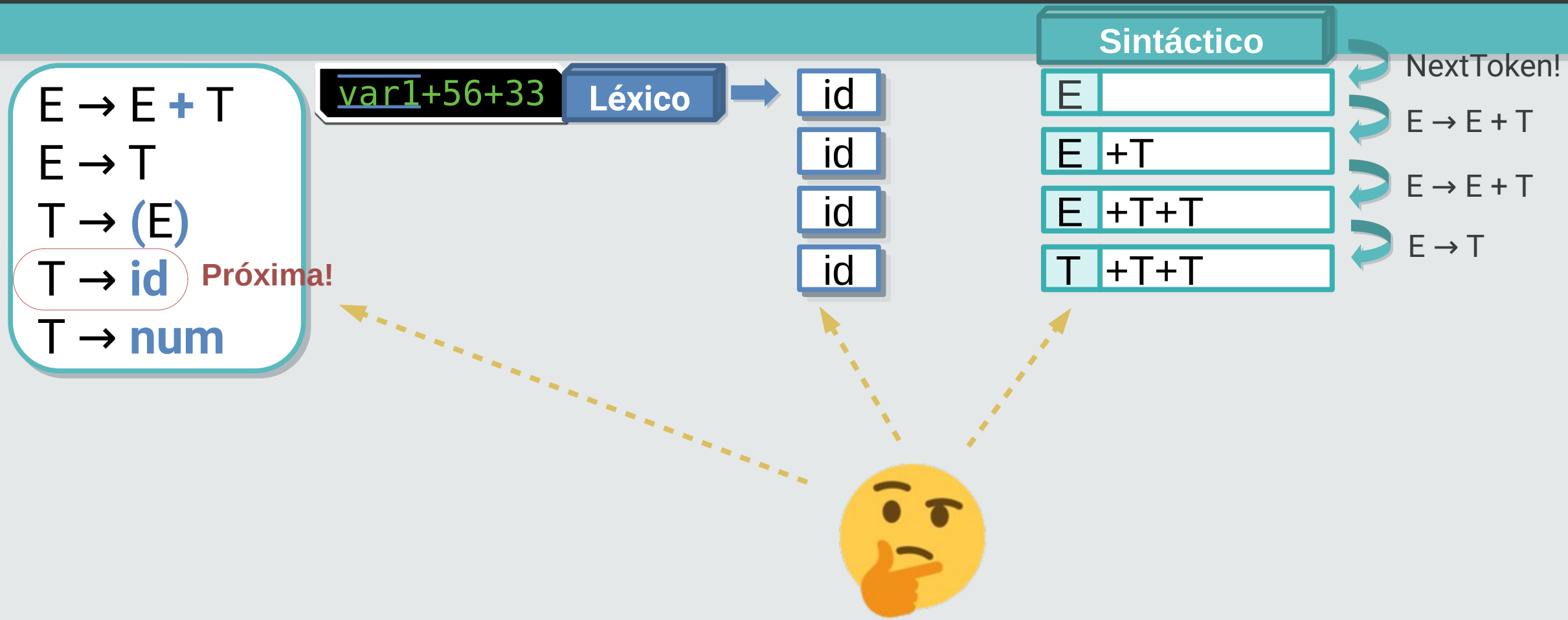
E → E + T
E → T
T → (E)
T → id
T → num



Sintáctico Descendente – Intuición – Ejemplo

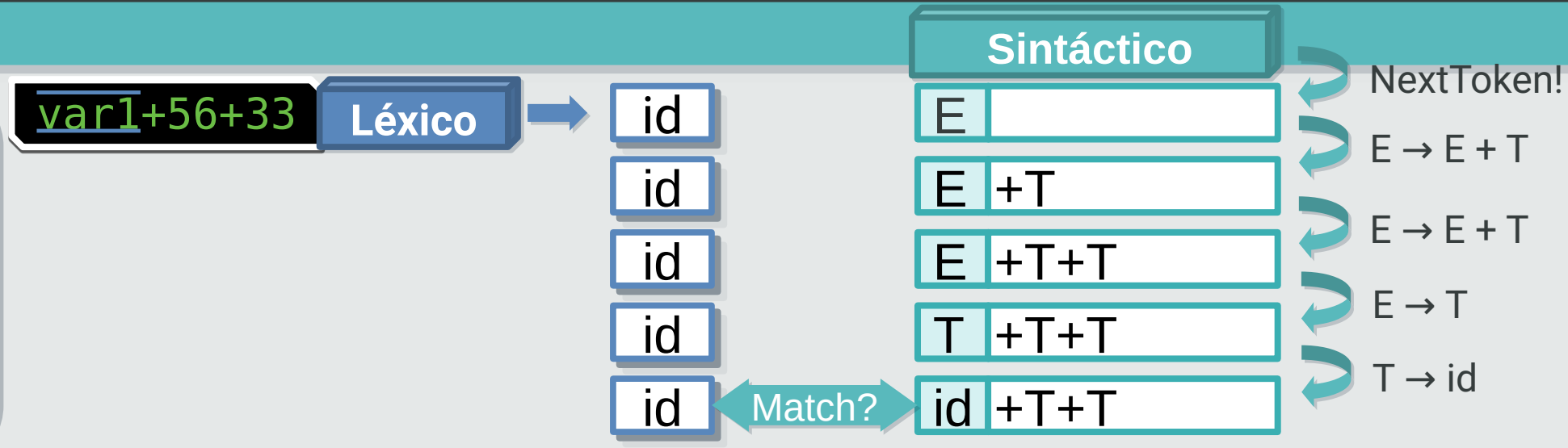


Sintáctico Descendente – Intuición – Ejemplo



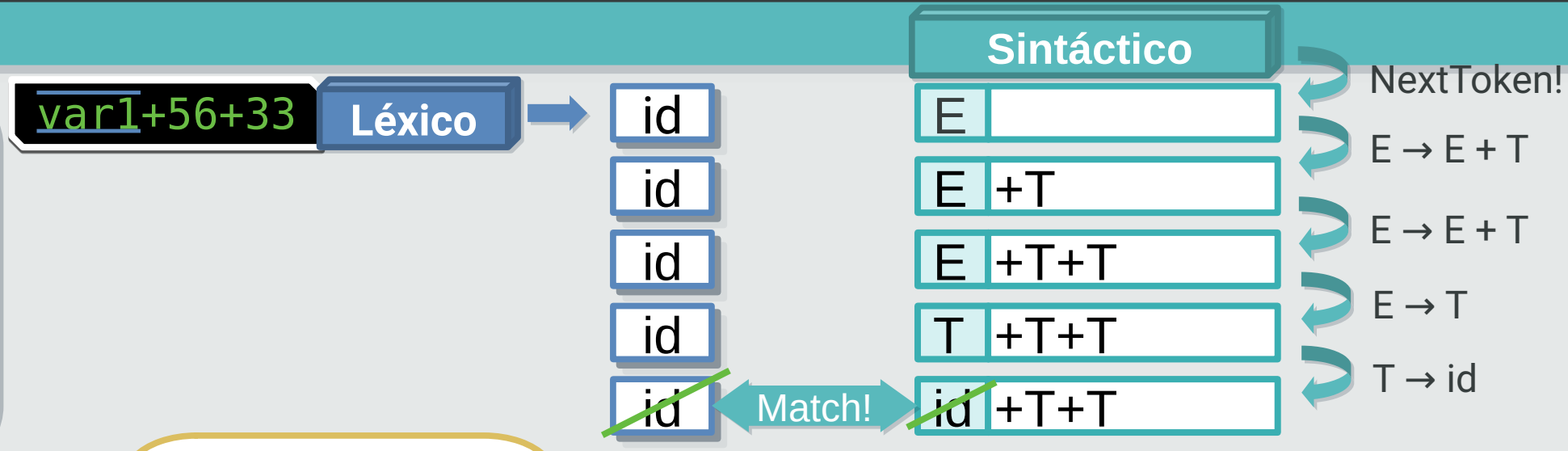
Sintáctico Descendente – Intuición – Ejemplo

E → E + T
E → T
T → (E)
T → id
T → num



Sintáctico Descendente – Intuición – Ejemplo

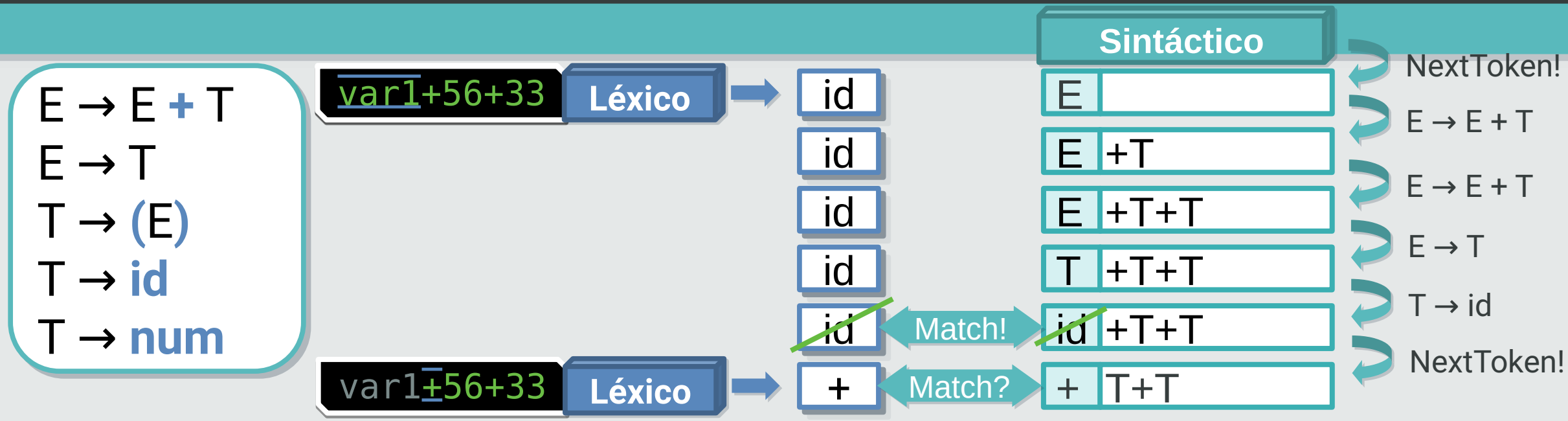
E → E + T
E → T
T → (E)
T → id
T → num



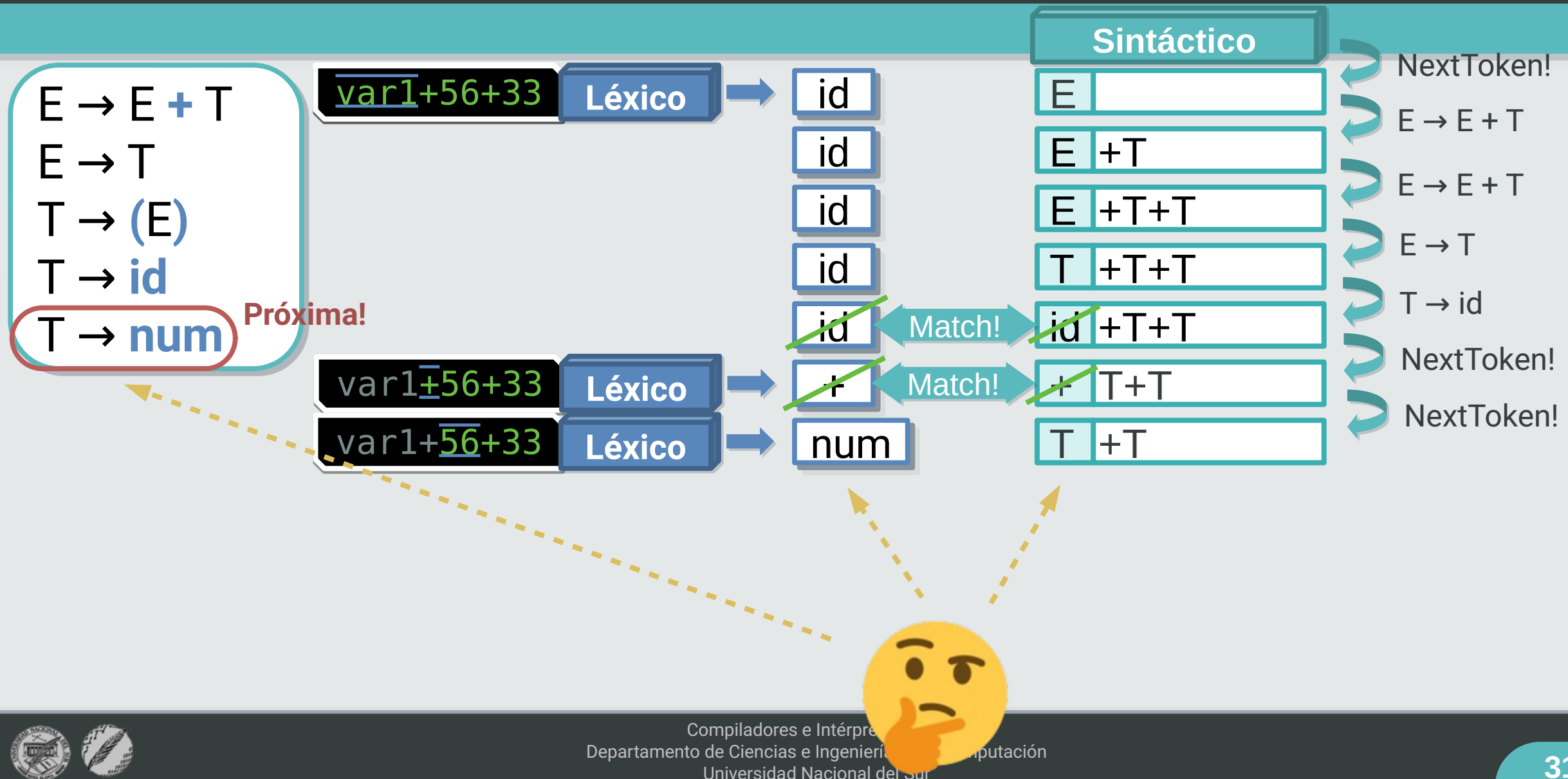
Ahora hay que pedir un nuevo token!



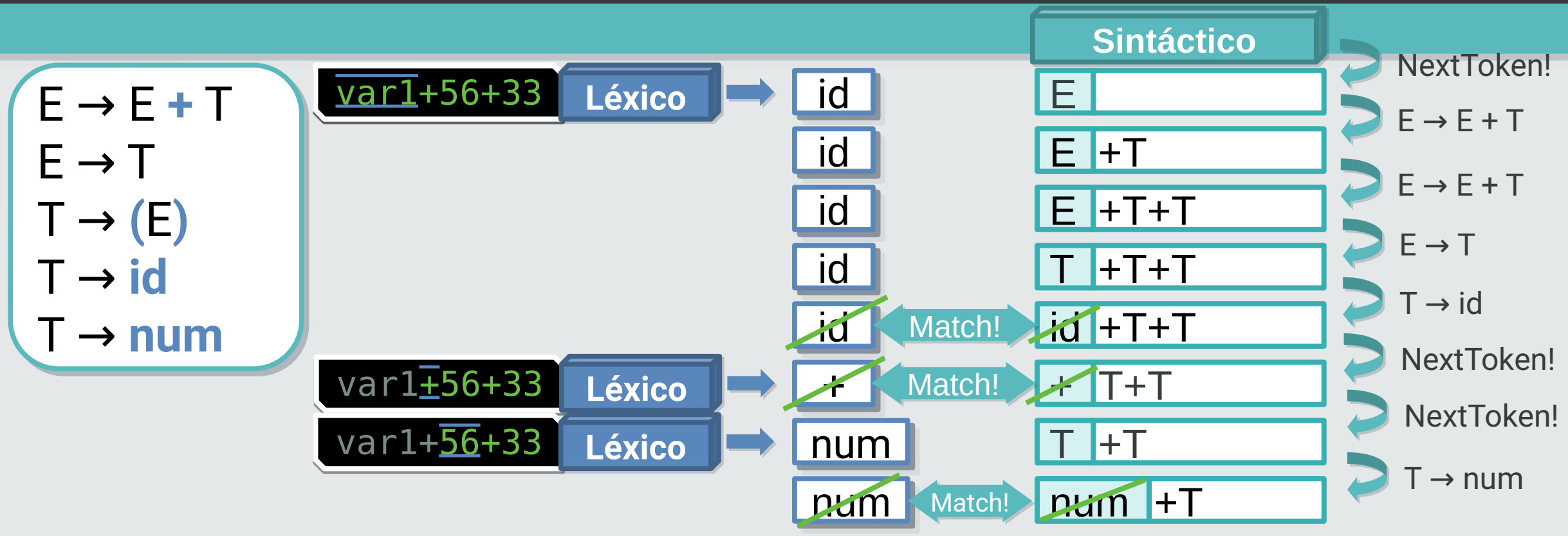
Sintáctico Descendente – Intuición – Ejemplo



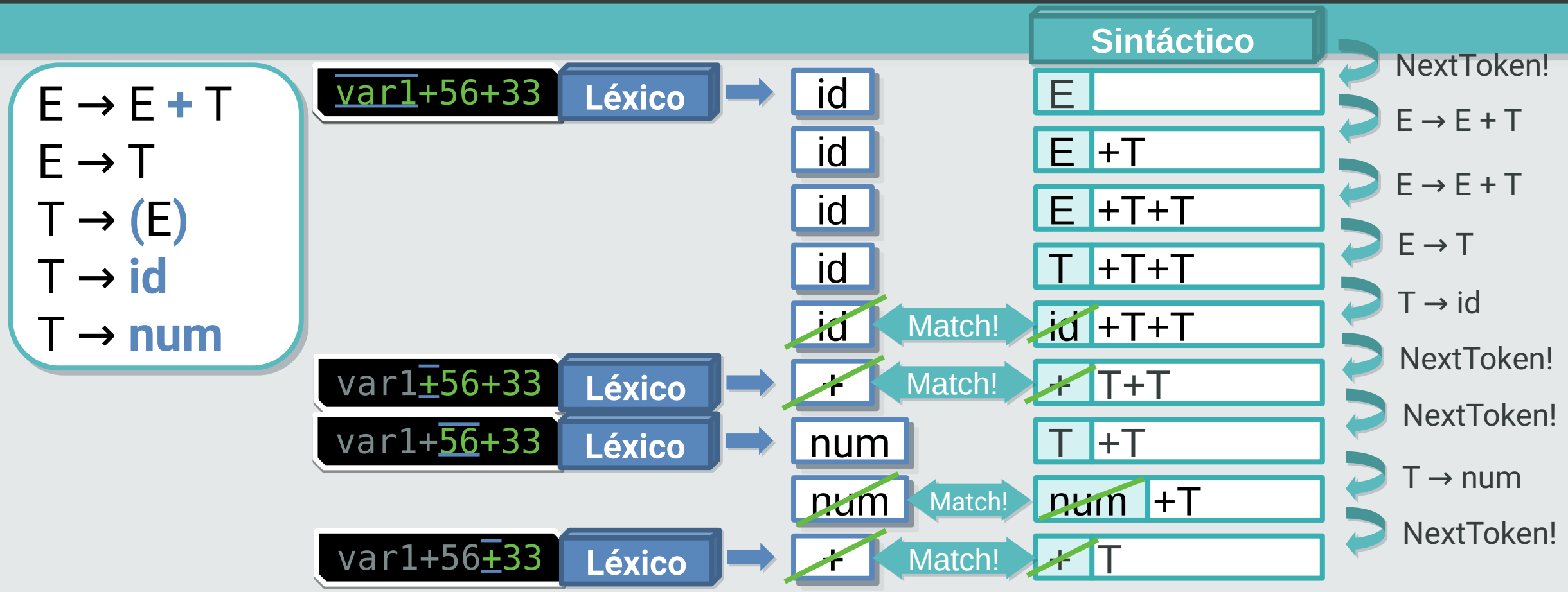
Sintáctico Descendente – Intuición – Ejemplo



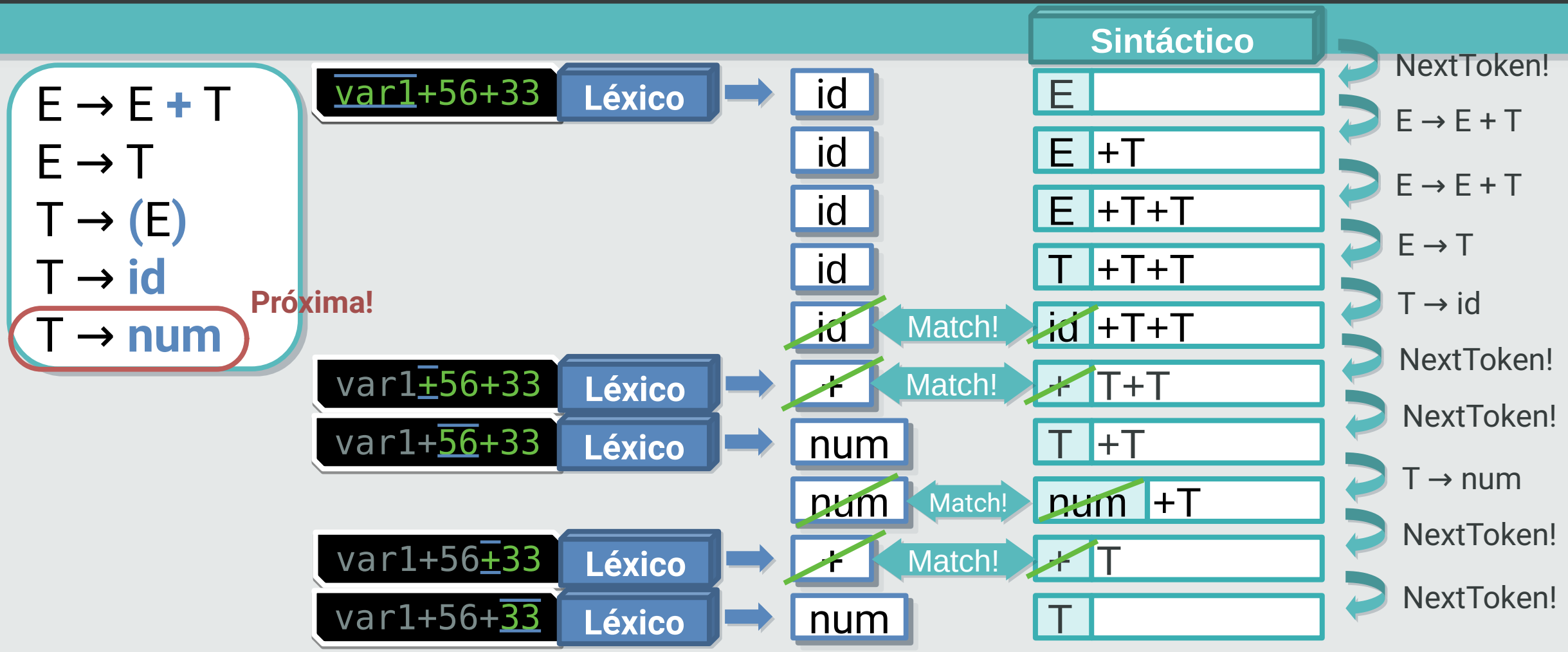
Sintáctico Descendente – Intuición – Ejemplo



Sintáctico Descendente – Intuición – Ejemplo

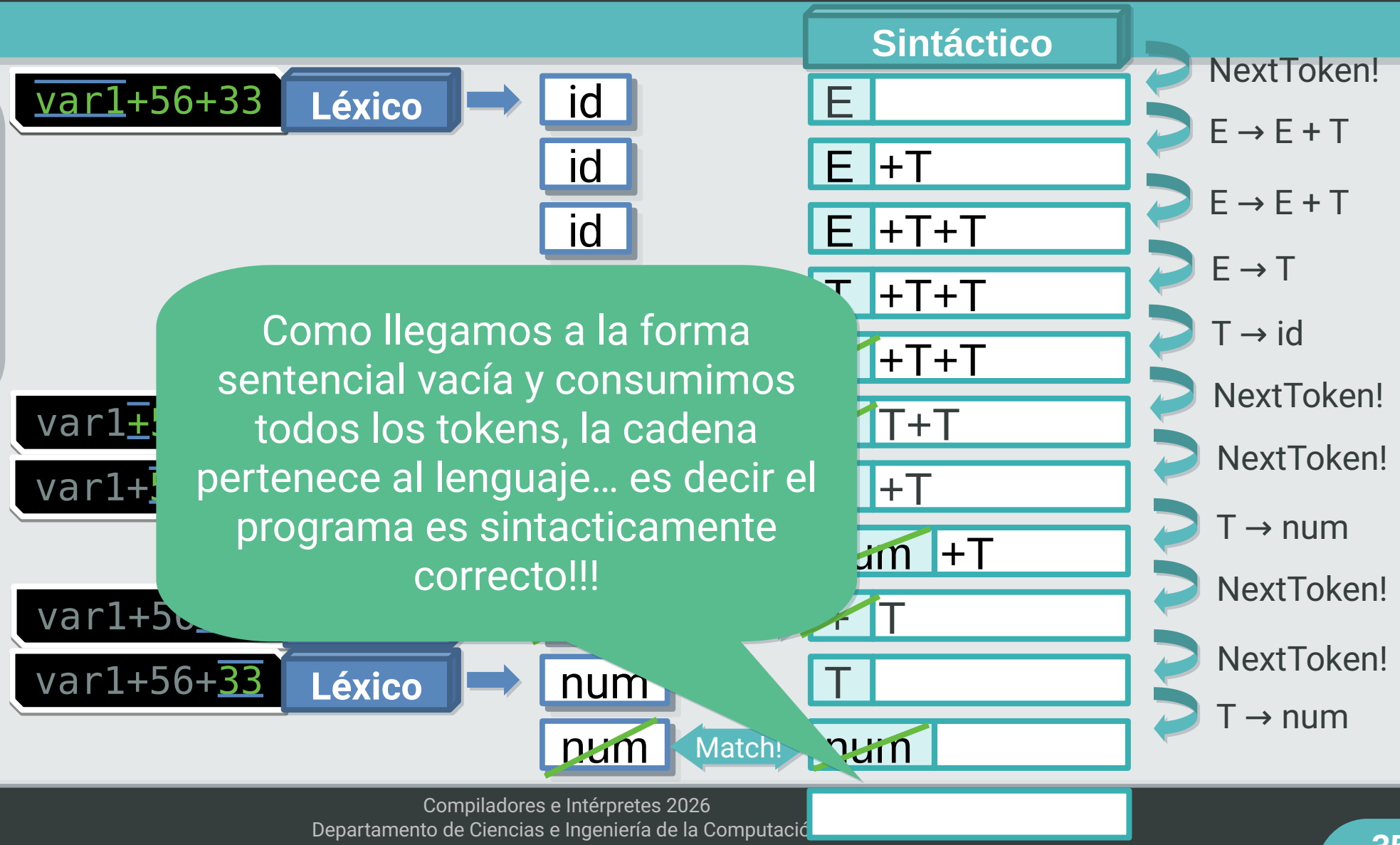


Sintáctico Descendente – Intuición – Ejemplo



Sintáctico Descendente – Intuición – Ejemplo

E → E + T
E → T
T → (E)
T → id
T → num



Sintáctico Descendente – Intuición – Ejemplo

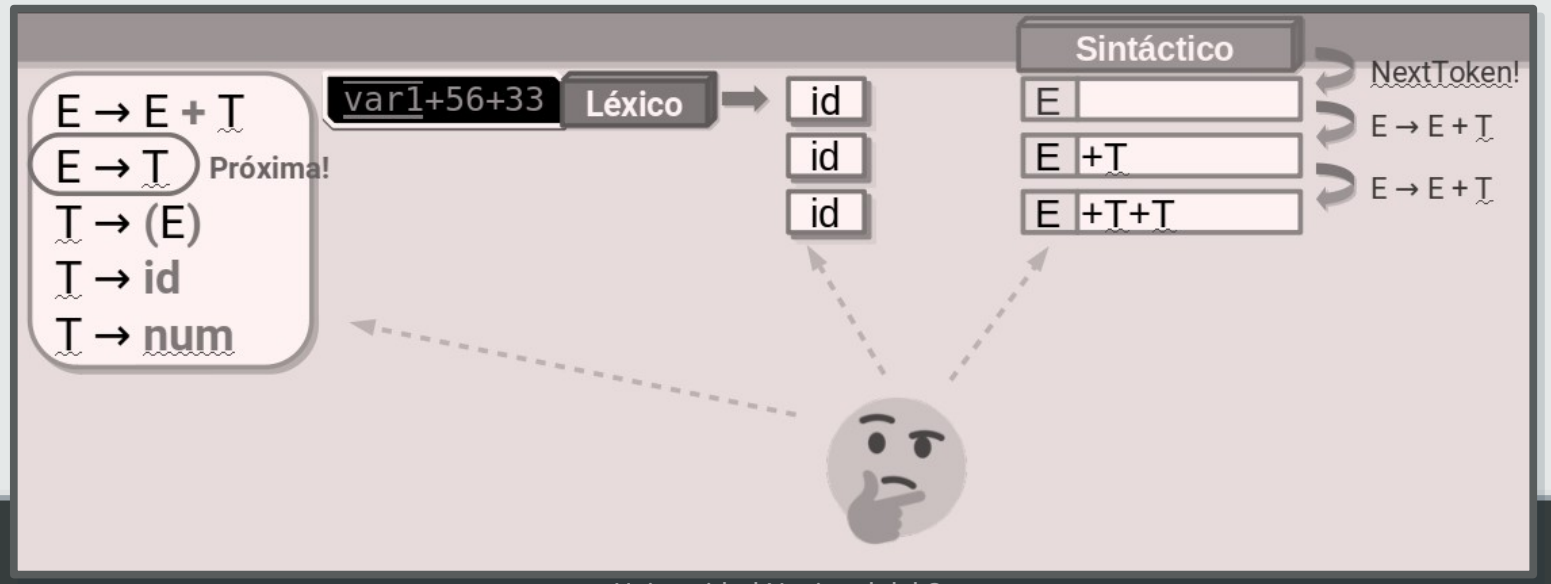
E → E + T
E → T
T → (E)
T → id
T → num



Para pensarlo un poco:
¿Cuando se produce un error sintáctico?¿Cómo nos daríamos cuenta utilizando esta estrategia?

¿Como Elegir la producción a aplicar?

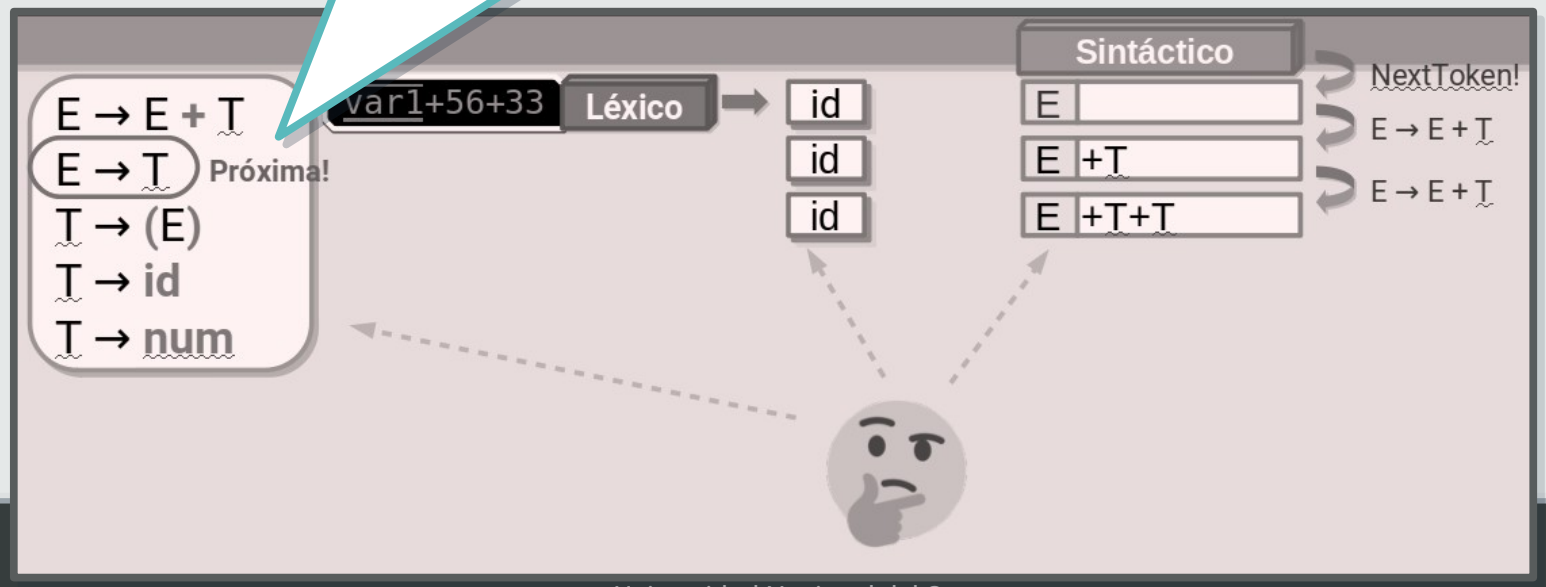
- La clave de la estrategia está en **cómo elegir las producciones** cuando estoy analizando un NT
- Claramente esto se tiene que hacer de forma **automatizada** y por eso hay varios **desafíos**… por ejemplo:



¿Como Elegir la producción a aplicar?

- La clave de la … **producciones** cuando estoy …
- Claramente est… **…tomatizada** y por eso hay varios d… **…s**… por ejemplo:

¿Por que elegimos esta producción en vez de E → E + T? Si a efectos prácticos estoy viendo el mismo NT y token que en el paso anterior en el cual habíamos elegido la otra

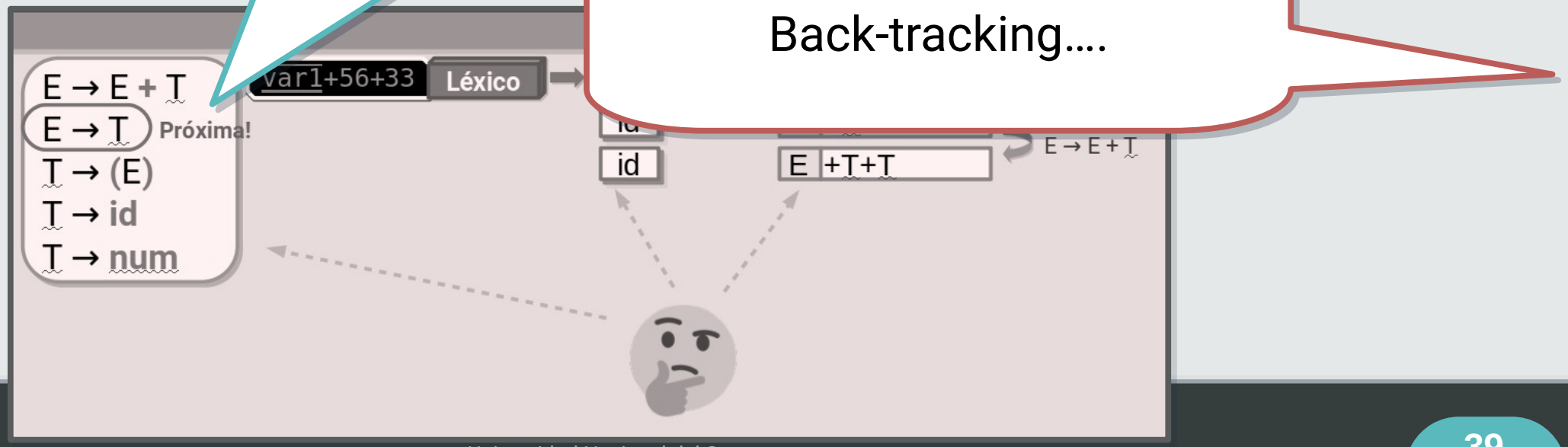


¿Como Elegir la producción a aplicar?

- La clave de la … producciones cuando estoy …
- Claramente est… tomatizada y por eso hay varios d… s… p…

¿Por que elegimos esta producción en vez de E → E + T? Si a efectos prácticos estoy viendo el mismo NT y token que en el paso anterior en el cual habíamos elegido la otra

Una alternativa es usar Back-tracking….

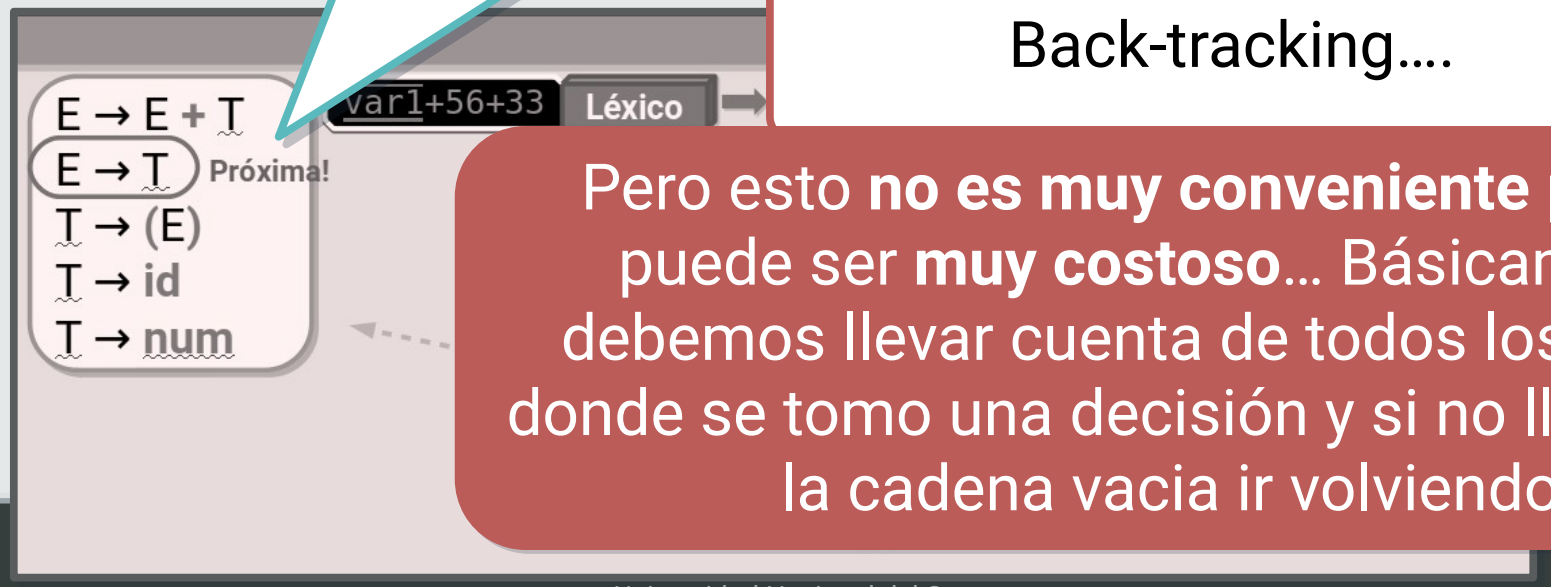


¿Como Elegir la producción a aplicar?

- La clave de la … producciones cuando estoy …
- Claramente est… tomatizada y por eso hay varios c… s… p…

¿Por que elegimos esta producción en vez de E → E + T? Si a efectos prácticos estoy viendo el mismo NT y token que en el paso anterior en el cual habíamos elegido la otra

Una alternativa es usar Back-tracking….



Pero esto **no es muy conveniente** por que puede ser **muy costoso**… Básicamente, debemos llevar cuenta de todos los puntos donde se tomo una decisión y si no llegamos a la cadena vacia ir volviendo

¿Como Elegir la producción a aplicar?

- Alternativa para el backtracking: utilizar **Predicción**
  - La intuición es utilizar los tokens actuales enviados por el léxico para elegir la producción adecuada cuando tengamos opciones
  - Las producciones de la gramática deben cumplir que
    - Dado un NT sus producciones tienen que tener la propiedad de que al **mirar k tokens siempre podemos elegir la correcta**

¿Como Elegir la producción a aplicar?

- Alternativa para el backtracking: utilizar **Predicción**
  - La intuición es utilizar los tokens actuales enviados por el léxico para elegir la producción adecuada cuando tengamos opciones
  - Las producciones de la gramática deben cumplir que
    - Dado un NT sus producciones tienen que tener la propiedad de que al **mirar k tokens siempre podemos elegir la correcta**

Si la gramática **no cumple** con esto debemos **adaptarla sin cambiar** el lenguaje que genera

¿Como Elegir la producción a aplicar?

- Alternativa para el backtracking: utilizar **Predicción**
  - La intuición es utilizar los tokens actuales enviados por el léxico para elegir la producción adecuada cuando tengamos opciones
  - Las producciones de la gramática deben cumplir que
    - Dado un NT sus producciones tienen que tener la propiedad de que al **mirar k tokens siempre podemos elegir la correcta**

Este tipo de gramáticas son llamadas **LL(k)**

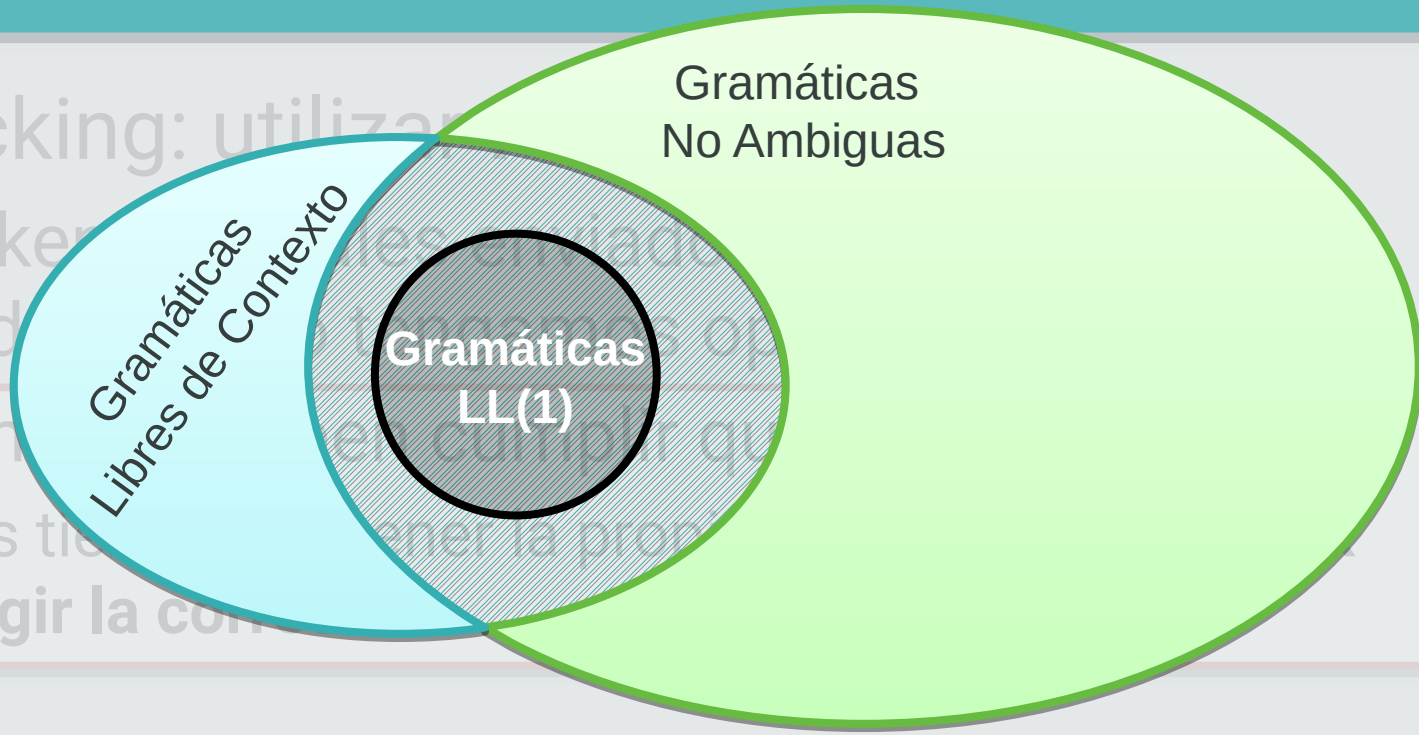
L: cadena de entrada se procesa de izquierda a derecha (**L**eft scannig)

L: derivación a izquierda (**L**eftmost derivation)

k: cantidad de símbolos de predición

¿Como Elegir la producción a aplicar?

Nosotros vamos a trabajar con gramáticas **LL(1)**, es decir con 1 símbolo/token de predicción



Este tipo de gramáticas son llamadas **LL(k)**

L: cadena de entrada se procesa de izquierda a derecha (**L**eft scannig)

L: derivación a izquierda (**L**eftmost derivation)

k: cantidad de símbolos de predición

Compiladores e Intérpretes

Analizador Sintáctico – Descendente – Gramáticas LL(1) – Primeros

Universidad Nacional del Sur
Departamento de Ciencias e Ingeniería de la Computación
2026

¿Como Elegir la producción a aplicar?

S → A$
A → B
A → C
...

Por ejemplo, en esta parte de la gramática, si queremos implementar el método para A... ¿Como elegimos entre cada producción?

¿Como Elegir la producción a aplicar?

S → A$
A → B
A → C
...

Por ejemplo, en esta parte de la gramática, si queremos implementar el método para A... ¿Como elegimos entre cada producción?

Solo mirando las producciones de A no alcanza... Tenemos que mirar las producciones de B y de C!

¿Como Elegir la producción a aplicar?

S → A$
A → B
A → C
B → xD
B → y
C → Ex
C → w
E → z
...

Supongamos que la gramática tiene estas otras reglas para B, C y E

¿Como Elegir la producción a aplicar?

S → A$
A → B
A → C
B → xD
B → y
C → Ex
C → w
E → z
...

La idea es que cuando estamos en A vamos a elegir la producción de A → B cuando el token actual es alguno de pueden aparecer primero en B...
En este caso x o y

Para A → C pasa algo similar, pero no nos alcanza con mirar las producciones de C tenemos que mirar también E...
En este caso van a ser w y z

¿Como Elegir la producción a aplicar?

S → A$
A → B
A → C
B → xD
B → y
C → Ex
C → w
E → z
...

La idea es que cuando estamos en A vamos a elegir la producción de A → B cuando el token actual es alguno de pueden aparecer primero en B...
En este caso x o y

¿Por que no x para A → C?

Para A → C pasa algo similar, pero no nos alcanza con mirar las producciones de C tenemos que mirar también E...
En este caso van a ser w y z

¿Como Elegir la producción a aplicar?

S → A$
A → B
A → C
B → xD
B → y
C → Ex
C → w
E → z
...

...estamos en A
...n de A → B
...alguno de
...o en B...

Porque, x nunca va a aparecer primero si vamos por C, ya si usamos C → w el primer token es w, y si usamos C → Ex el primer token depende de E en este caso z

¿Por que no x para A → C?

Para A → C pasa algo similar, pero no nos alcanza con mirar las producciones de C tenemos que mirar también E...
En este caso van a ser w y z

¿Como Elegir la producción a aplicar?

S → A$
A → B
A → C
B → xD
B → y
C → Ex
C → w
E → z
...

Si el token actual es **x** o **y**, entonces reemplazo por B,
Sino si el token actual es **w** o **z** reemplzazo por C,

¿Como Elegir la producción a aplicar?

S → A$
A → B
A → C
B → xD
B → y
C → Ex
C → w
E → z
...

Si el token actual es **x** o **y**, entonces reemplazo por B,
Sino si el token actual es **w** o **z** reemplzazo por C,
Sino Error!

¿Como Elegir la producción a aplicar?

S → A$
A → B
A → C
B → xD
B → y
C → Ex
C → w
E → z
...

Si el token actual es **x** o **y**, entonces reemplazo por B,
Sino si el token actual es **w** o **z** reemplzazo por C,
Sino Error!

Entonces, cuando tenemos mas de una producción tenemos que mirar los "primeros" terminales/tokens a los que nos puede llevar cada producción...
Analicemos esta noción de **primeros**

Primeros

Primeros

β es una secuencia que puede tener terminales y no terminales

- En General:

  Consideremos una producción de la forma Y → β y un token **x**

Primeros

β es una secuencia que puede tener terminales y no terminales

- En General:

  Consideremos una producción de la forma Y → β y un token **x**

Si a partir una forma sentencial β podemos llegar mediante derivación a otra forma donde el *terminal* **x** aparece primero decimos que:

x esta en Primeros(β)



Primeros

β es una secuencia que puede tener terminales y no terminales

- En General:

  Consideremos una producción de la forma Y → β y un token **x**

Esta noción, captura las intuiciones que veníamos viendo antes y va a ser central para determinar directamente como elegir que producción de un NT vamos a aplicar…
Antes de definirla formalmente veamos algunos ejemplos

Si a partir una forma sentencial β β… derivación a otra forma donde el *termin*… aparece primero decimos que:

x esta en Primeros(β)

Primeros



Intuitivamente Primeros(β) nos da el conjunto de todos los **terminales** que pueden aparecer **al principio** de las formas resultantes de **derivaciones de β**, Por ejemplo:

L → D L | **$**
D → A | M
A → V T **id**
M → T **id(**P**)**
V → **priv** | **pub**
T → **int** | **char**
P → T **id** P | ε

Primeros(**id**) = ?
Primeros(V) = ?
Primeros(VT**id**) = ?
Primeros(T**id(**P**)**) = ?
Primeros(A) = ?
Primeros(DL) = ?
Primeros(L) = ?

Primeros



- Si por alguna razón β nos lleva a vacío (ε) por completo, entonces e esta en Primeros(β)

L → D L | **$**
D → A | M
A → V T **id**
M → T **id**(P)
V → **priv** | **pub**
T → **int** | **char**
P → T **id** P | ε

Primeros(ε) = ?
Primeros(P) = ?
Primeros(P**id**)= ?
Primeros(PT) = ?
Primeros(PP) = ?
Primeros(PV) = ?

Primeros

- Si por alguna razón β nos lleva a vacío (ε) **por completo**, entonces e esta en Primeros(β)

L → D L | **$**
D → A | M
A → V T **id**
M → T **id**(P)
V → **priv** | **pub**
T → **int** | **char**
P → T **id** P | ε

Primeros(ε) = {ε}
Primeros(P) = {**int, char,** ε}
Primeros(P**id**)= {**int, char, id**}
Primeros(PT) = {**int, char**}
Primeros(PP) = {**int, char,** ε}
Primeros(PV) = {**int, char, priv, pub**}

Un no terminal X es vaciable si X ⇒* ε

Primeros

- Si por alguna razón β nos lleva a vacío (ε) **por completo**, entonces e esta en Primeros(β)

L → D L | **$**
D → A | M
A → V T **id**
M → T **id(**P**)**
V → **priv** | **pub**
T → **int** | **char**
P → T **id** P | ε

Primeros(ε) = {ε}
Primeros(P) = {**int, char,** ε}
Primeros(P**id**)= {**int, char, id**}
Primer
Primer
Primeros(P V) = {**int, char, priv, pub**}

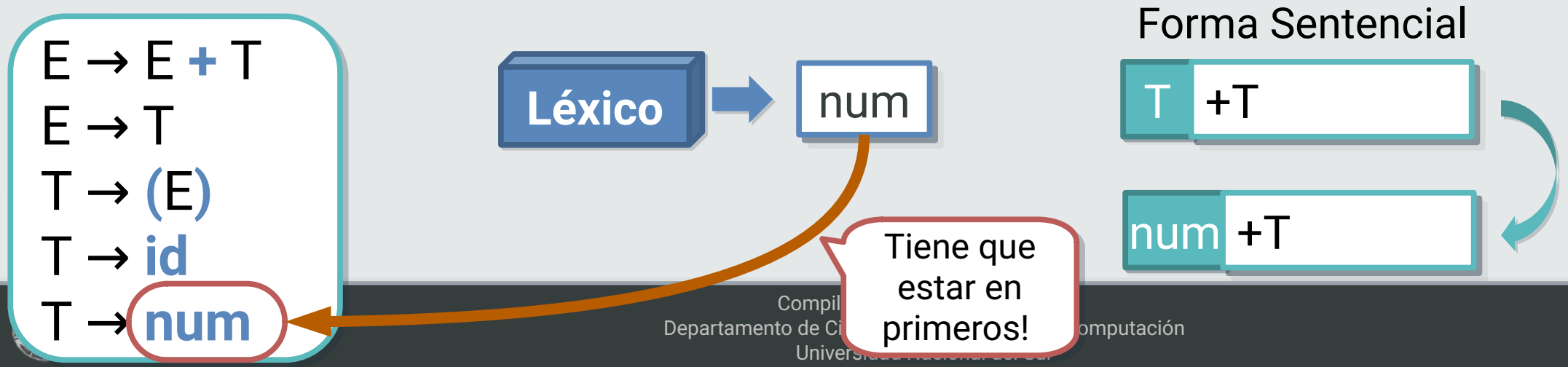
¿Hay algún símbolo vaciable en la gramática del ejemplo?

P

Un no terminal X es vaciable si X ⇒* ε

Usando Primeros en la Estrategia Descendente

- Intuición: **Simular** la derivación a izquierda
  - Emular la forma sentencial de cada paso de la derivación, analizando el **símbolo** (T o NT) **de más a la izquierda** y el **token actual** enviado por el Léxico
  - Si el símbolo de más a la izquierda de la forma es:
    - **NT**: **elegir** una **producción** para ese no terminal **que tenga al token actual en primeros de su cuerpo** y **reemplazarlo por la parte derecha**



Gramaticas LL(1) y Primeros

Primeros y LL(1)

- Cualquier gramática LL(1), para un NT con producciones de la forma

$$A \rightarrow \beta 1 \mid \beta 2 \mid \ldots \mid \beta n$$



Primeros y LL(1)

- Cualquier gramática LL(1), para un NT con producciones de la forma

$$A \rightarrow \beta 1 \mid \beta 2 \mid \ldots \mid \beta n$$

- Verifica que, para cualquiera $i \neq j$:

$$Primeros(\beta i) \cap Primeros(\beta j) = \emptyset$$



Primeros y LL(1)

E → E **+** T
E → T
T → **id** | P | **num**
P → **id**(E)

Primeros(P) = ?
Primeros(**id**) = ?

Primeros y LL(1)

E → E **+** T
E → T
T → **id** | P | **num**
P → **id**(E)

Primeros(P) = {**id**}
Primeros(**id**) = {**id**}
Primeros(T) = ?
Primeros(E **+** T) = ?